2IV60 assignments

Q1, Academic year 2025-2026

1 Introduction

For the practical component of 2IV60, you are going to implement some well-known computer graphic concepts, namely, morphing, shading, environment mapping, splines, and volume rendering. You will realize this through so-called GLSL shader programs that run in a shader development environment, called SHADERed.

2 Prerequisites & prior knowledge

The text of this assignment does not pretend to be complete. If you feel that some of the material is unclear, you need to check out the material that we make available in Canvas. There you will find at least the following items.

1. SHADERed program and links to online documentation;
2. Our SHADERed tutorial;
3. Lecture slides;
4. Links to online resources regarding GLSL.

3 Modus operandi

- The assignment is done in groups of exactly two students.
- The assignment is subdivided in exercises. The results of the exercises marked with the canvas logo are to be submitted through Canvas, among others for grading and fraud detection.
- Each exercise is graded based on the following:
  - The expected approach as described in this document;
  - The criteria given in the corresponding Canvas rubric;
  - Sufficient, clear, and, correct documentation in the shaders;
  - Correct visual output.

4 *Week 2, Exercise 1/2: Morphing*

MORPHING is a technique to change the shape of an object using continuous animation. A morphing effect can be achieved if a vertex shader computes a time-dependent vertex position[1].

[1] Remember that SHADERed has a uniform variable that keeps the time.

Open the project called morph.sprj. Modify the skeleton vertex shader so that the morphing effect works on the given *unit sphere*[2], as shown in the morphing demo video in Canvas.

[2] Sphere with radius 1, centered around the origin.

First, make the sphere visible:

- Compute the Model-View-Projection matrix in function main() using the system variables, representing Geometry/Modeling and View-Projection transformation matrices.
- Compute gl_Position, using the Model-View-Projection matrix and vector p. The variable gl_Position represents the position of the current vertex in *homogeneous* coordinates.

Second, implement the morphing:

- Do the computations in Model space.
- Start with computing the intersection[3] of the unit cube[4] with the half-line starting in the origin and incident with the current vertex.
- Convert intersection into an offset of the cube face at hand using some periodic function.
  - Complete function offset () by assigning to variable limit[5]. And by assigning a suitable value to the variable bias [6].

[3] Use preparation exercise in section 9.

[4] A unit cube has dimensions $[-1,1] \times [-1,1] \times [-1,1]$.

[5] Think about what variable restriction actually represents.

[6] You can leave this step for now, and once the animation is working, you can start experimenting to find a suitable bias.

5 *Week 2, Exercise 2/2: Phong Shading*

PHONG SHADING is a shading technique based on normal vector interpolation resulting in more realistic highlights and reduced Mach banding.

■ Open the project called phong.sprj and implement Phong shading by adapting both the vertex shader and the fragment shader.

- Compute in the vertex shader for a given vertex and normal in model space, the vertex position P and normal vector N in eye space, using the camera's view matrix mat4 matV, and the modelling transformation matrix mat4 matGeo.
- Use the interpolated input for the fragment shader, P and N, in the shading function to compute the actual Phong shading[7] when given also a light source and a material specification.

```
vec4 shading(vec3 P, vec3 N, LightSource light, Material m)
```

[7] The details can be found in the shading lecture slides.

6 *Week 3, Exercise 1/1: Environment mapping*

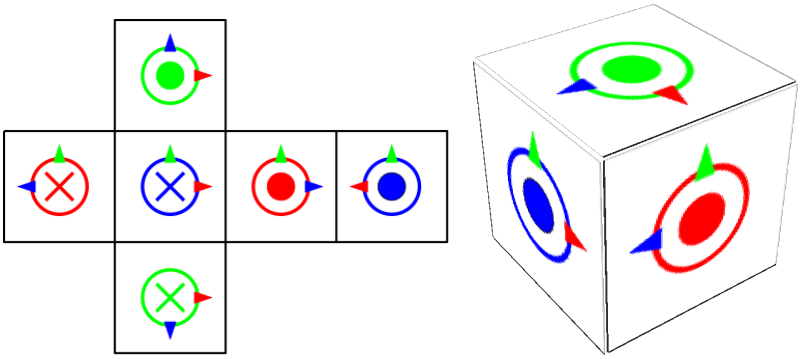


Figure 1: Test cube map. *left*: with encoding for the axes: x, y, and z are encoded as red, green, and blue, respectively; triangles and dots denote positive axes, crosses negative axes. *right*: folded to a cube.

ENVIRONMENT MAPPING is a long-standing texture mapping technique used to enhance the realism of a rendered scene by taking into account the reflection of the environment in an object. Here we consider CUBE MAPPING[8] where the environment is a unit cube[9] (in Model space) textured with a cube map, a texture in the shape of an unfolded cube; see figures 1 and 2.

Open the project called environment.sprj. Implement environment mapping using a vertex and a fragment shader.

Follow the remarks in the given template code for the shaders. Furthermore, in the vertex shader consider the following.

- Assign to vec3 Q the Cartesian coordinates of the shader's input vertex vec4 P.
- Compute the eye position vec3 E in Cartesian Model space coordinates[10].
- Compute vec3 I, the vector from P towards the camera, in Cartesian Model coordinates.
- Finally, reflect vector vec3 I in input normal vec3 N and assign the result to vec3 R.

The vector varying vec3 R is interpolated and passed to the fragment shader to compute a proper texture coordinate in a cube map:

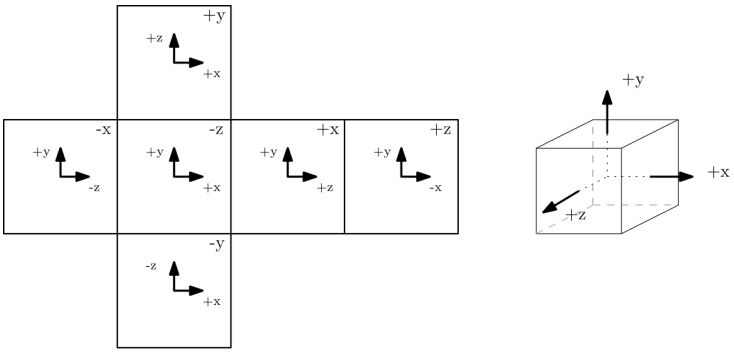


Figure 3: (left) Unfolded cube map texture showing x, y, z axes orientations on its faces; (right) The corresponding cube and the axes in 3D.

[8] See slides on environment mapping.

[9] Again: $[-1,1] \times [-1,1] \times [-1,1]$.

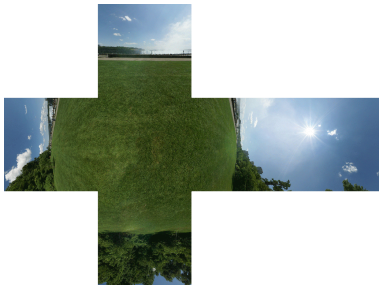


Figure 2: Sample cube map.

[10] *Hint:* The eye position is located in the origin in Eye space.

- First, compute the intersection vec3 h of the reflected vector R with the unit cube[11].
- Transform the intersection point to coordinates $(u, v) \in [-1, 1] \times [-1, 1]$ on the intersected side of the cube: Avoiding case distinction as much as possible, define a transformation matrix to map (u, v) to $[1, 0] \times [0, 1]$ coordinates in the cube map. Do so by building matrices for (1) mapping[12,13] (u, v) on $[0, 8] \times [0, 6]$ using local axes (position and orientation) as shown in Figure 3, and (2) mapping $[0, 8] \times [0, 6]$ to $[0, 1] \times [0, 1]$.

[11] Do so in the same manner as in the Morphing assignment.

[12] Follow along the preparation exercise in section 10 to help you setup the appropriate transformations.

[13] Use function mat3 Mat() to construct a 3×3 homogeneous matrix from given columns.

7 week 5, Excercise 1/1: Splines in a geometry shader

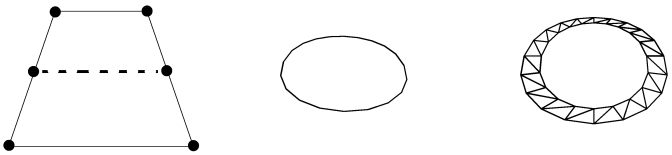


Figure 4: Eight control points (vertices of two adjacent quads) being transformed into a curve and a surface, using geometry shaders.

A *geometry shader* is an optional shader stage between the vertex and the fragment shader. The geometry shader takes a geometric primitive, consisting of a number of vertices, as input. The shader can then transform the input into a new primitive, consisting of different vertices. A good example for using a geometry shader is the interpolation of curves, like Bézier curves.

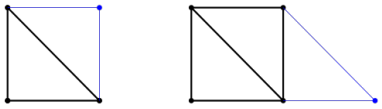


Figure 5: Construction of a triangle strip. Each generated vertex (blue) forms a triangle with the last two vertices.

In this exercise you implement a GLSL effect that constructs a cubic Bézier curve when given four control points – the vertices of a quad. You give the curve a certain width and height to form a track. In a vertex shader you animate a cube that moves along the track. Finally, you change the track into a Möbius band. See video and figure 4. **We *require* you to use matrices in solving sub problems.**

You are given a skeleton project that displays a quad. In the geometry shader, vertices are generated using GLSL function EmitVertex(), see function genVertex().

1. Construct a Bézier curve given a quad in $z = 0$ plane as input in the geometry shader.
   - First, implement the curve functions evalBezier(float t,mat4 G) and evalBezierTan(float t,mat4 G) to return a point on the cubic curve[14] at parameter t and its tangent, respectively. G is the so-called geometry vector that contains a column of control points (row vectors).
   - Next, make the shader emit vertices along the curve. The amount of segments per curve is defined by the variable uNum. The end results should look similar to the curve in the middle image of figure 4 [15].
2. Replace the curve by a track symmetric in $z = 0$ with width $2D$ and height $2H$ by constructing 4 triangle strips.
   - First generate[16] a triangle strip[17] for the top of the track by computing a local orthonormal axis system mat4 Orient at point $p(t)$ of the curve with x-axis perpendicular to the curve, y-axis tangent to the curve, and z-axis equal to $(0, 0, 1)^T$. Define the points of the triangle strip in these local axis systems.

[14] The four control points are available as gl_in[i].gl_Position with $i = 0, 1, 2, 3$.

[15] Note that you need two quads to come to that result.

[16] In the layout of the geometry shader use triangle-strip instead of line-strip.

[17] The construction of the triangle strip is as shown in figure 5.

- Similarly[18], generate triangle strips for the two sides and the bottom of the track. Color the sides in a distinguishing color.

3. Animate a cube along the track.

   - Implement the vertex shader of shader pass Anim to move the cube along the curve. (1) Activate the Anim shader pass by clicking eye icon next to it. (2) Implement[19] the evalBezier and evalBezierTan functions. (3) Implement the translation and scaling transformation functions, T(vec3 t) and S(vec3 s). (4) Using those matrices and mat4 Orient construct a local orthornormal axis system in which the $[-\frac{1}{2},\frac{1}{2}]\times[-\frac{1}{2},\frac{1}{2}]\times[-\frac{1}{2},\frac{1}{2}]$ can be drawn to result in the animated cube whose positive y-face is in the tangent direction of the curve.

4. Replace the track by a Möbius band.

   - Use a mat4 Twist transformation to change the track generation in the geometry shader and the animation in the vertex shader accordingly.
   - Add a switch in the GUI to switch the Möbius band on and off.

[18] If you did it correctly, you should see that the only difference between the 4 triangle strips are the chosen local coordinates, making this an ideal candidate for implementation with a function with two points given in local coordinates as parameters.

[19] Reuse code from the geometry shader, but modify to use the control points $P0, P1, P2, P3$ as parameters.

8 Week 7, Exercise 1/1, Path Tracing

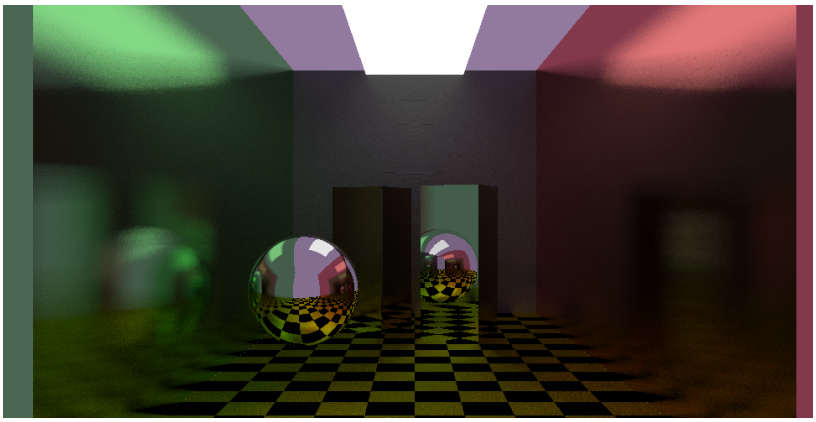


Figure 6: Path tracer with purple sky, white area light and metallic/glossy objects rendered with, sometimes fuzzy, specular reflections.

RAY TRACING simulates light by tracing rays from the camera into a scene, accurately modeling reflections and shadows using geometric precision. It typically follows deterministic paths and simplified lighting models. Traditional implementations rely on recursion, which is not available in GLSL.

PATH TRACING introduces randomness to ray behavior, allowing rays to bounce and sample light stochastically. This produces noise that must be reduced by averaging many samples. While full path tracing can capture global illumination effects, our implementation focuses on specular reflections with perturbations, omitting diffuse lighting and transparency.

In this assignment you are going to build a GPU-accelerated path tracing using GLSL; see figure 6 for a possible outcome. You are given a skeleton code in which you have to implement, among others, intersection algorithms and tracing of rays. The intersection computations have to be done in model space. The scenes you are going to render consist of transformed *unit objects*: unit spheres[20] and unit cubes[21]. These objects are placed into the world by applying a homogeneous transformation matrix TRS, which combines translation (T), rotation (R), and scaling (S).

[20] a sphere centered at the origin with radius 1

[21] a cube centered at the origin with side length 2

The fragment shader of the skeleton code already contains most important functionality for this exercise. In the main function, function computeColor is called. Inside, calculateFinalColor is called a number of times, determined by averageCount. The resulting colors after these calls are averaged and returned by function computeColor. Function calculateFinalColor casts a ray with origin rayOrigin and direction rayDir, which is checked for intersections with objects in the scene by the AllObjectsRayTest function. The function returns a hitData structure containing information about the intersection point on the nearest object hit by the ray; see comments in the skeleton code. Depending on the object that is hit, the loop is executed again with a newly computed ray. This continues until the ray hits a light source[22] or reaches

[22] The sky is a light source that is modeled as a big sphere around the whole scene. So, there is always a hit.

its maximum amount of bounces. Every time a ray bounces off of something, the absorbMul variable is decreased so as to simulate light absorption. The initial scene consists of a ground floor, modeled as a box, and a sphere. For something to appear in the preview window, several things need to be done:

- First, the functions rayUnitSphereIntersect and rayUnitCubeIntersect should be implemented. Afterwards, it should be possible to see a unit sphere and a unit cube[23].
- The objects are not yet transformed. So, next fix the transformation matrices in the makeObject function.
- Fix the AllObjectsRayTest to find the closest object; follow the steps indicated in the code. You now should have something as in Figure 7.
- Next, in calculateFinalColor, remove the line that prematurely ends the loop over the bounces and update rayOrigin and rayDir to comply with the ray after bouncing off of the object that has been hit. See Figure 8 for the result.
- Up to this point, the ray tracing is deterministic. Modify it by introducint a random perturbation to the computed reflection vector. To see the effect play with the averageCount parameter in the pinned properties of shaderEd. Make the effect also scalable by using the roughness parameter of objects.
- Next, construct a scene to showcase your raytracer (see demo video on canvas): at a minimum include local light source(s) [24], add animation(s) and make use of uniform variables that can be set in the pinned properties in shaderEd.
- For the enthusiasts: To further enhance this raytracer, you may, for instance, add diffuse reflections[25], environment mapping, transparency, and objects defined by implicit functions, or make enhancements to speed up the ray tracer.

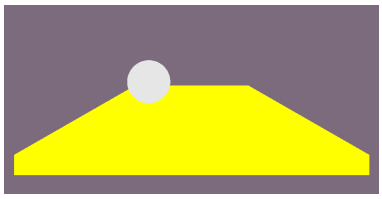


Figure 7: Intermediate result: white ball, yellow plane, purple sky.

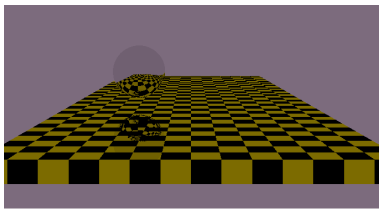


Figure 8: Intermediate result: Fully specular sphere and plane.

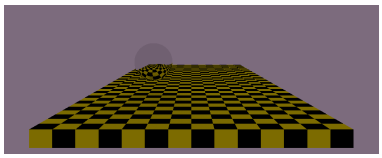


Figure 9: A shiny sphere with a much less shiny plane.

[23] Note that the cube might hide the sphere.

[24] refer to the sky object for guidance

[25] This can be achieved by randomly deciding at each bounce whether the next interaction is specular or diffuse. In the specular case, the ray is reflected according to the reflection law (with possible perturbations). In the diffuse case, a new ray direction should be chosen randomly according to an appropriate distribution.

9 *Appendix: Morphing shader*

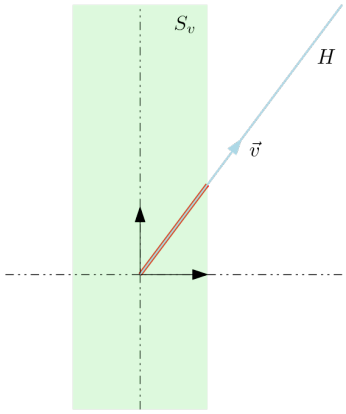


Figure 10: Vertical slab S_v in green; half line H in blue with direction vector $\vec{v}$; intersection of half-line H and slab S_v in red.

We consider the intersection of objects with a half-line. A half-line is often used in computer graphics, for instance to model light rays. Here we consider half-lines and objects in $\mathbb{R}^2$. Let H be a half-line starting in the origin with direction $\vec{v}$:

$$H = \left\{ \lambda \cdot \vec{v} \,\middle|\, \lambda \geq 0 \right\} \text{ with } \vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

Furthermore, we consider objects, called slabs. Slabs are infinitely-long strips. We consider both a horizontal and a vertical slab:

$$S_h = \left\{ \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^2 \,\middle|\, -1 \leq y \leq 1 \right\} \;\wedge\; S_v = \left\{ \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^2 \,\middle|\, -1 \leq x \leq 1 \right\}.$$

1. For $v_x \neq 0$ compute the intersection $H \cap S_v$ of the half-line with the vertical slab. Give the λ-interval on H for this intersection; avoid case distiction in the (final) expressions for upperbound and lowerbound.
2. Use slabs to construct a (filled) unit-square $S = [-1, 1] \times [-1, 1]$.
3. Give the λ-interval on H for the intersection $H \cap S$ of the half-line and the square
4. What if $v_x = 0$, $v_y = 0$, or both?
5. What is the intersection point $\vec{i}$ of H and the border of the square S.
6. What is the outward normal vector $\vec{n}$ on the square in intersection point $\vec{i}$.

10 Appendix: Environment mapping shader

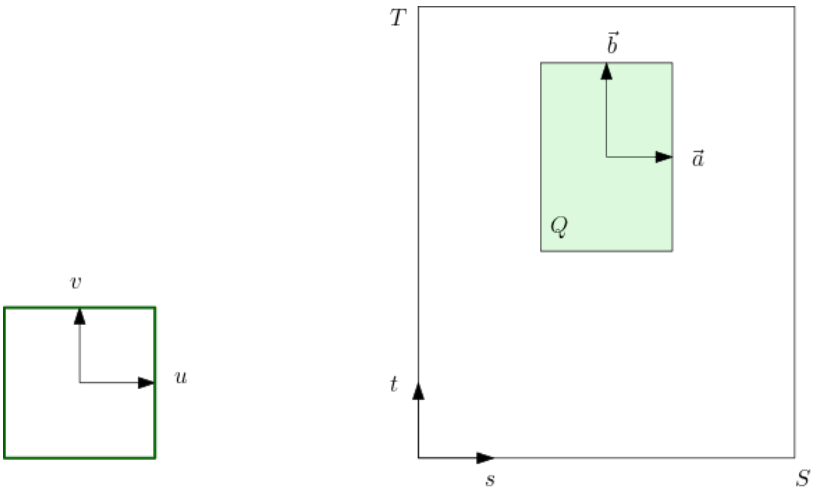


Figure 11: *left*: parameter space with $u, v \in [-1, 1]$; *right*: texture map with $s \in [0, S]$ and $t \in [0, T]$; Q is a quadrangle with axis $\vec{a} = (a_x, a_y)$ and $\vec{b} = (b_x, b_y)$.

Consider the side at $y = 1$ of a unit cube $[-1, 1] \times [-1, 1] \times [-1, 1]$. We like to map each point of this side to a color in a texture map $[0, S] \times [0, T] \rightarrow Color$.

1. Give a parametric description $P(u, v)$ for this side of the cube with parameters $u, v \in [-1, 1]$.

2. Construct a homogeneous matrix H that maps parameter points (u, v) to texture coordinates (s, t) such that the texture map is scaled to fit exactly, but **possibly** with distortion.

3. Construct a homogeneous matrix H that maps parameter points (u, v) to a quadruple Q in texture coordinate space, given by

$$Q = \left\{ (p_x, p_y) + \alpha(a_x, a_y) + \beta(b_x, b_y) \middle| -1 \leq \alpha, \beta \leq 1 \right\}.$$